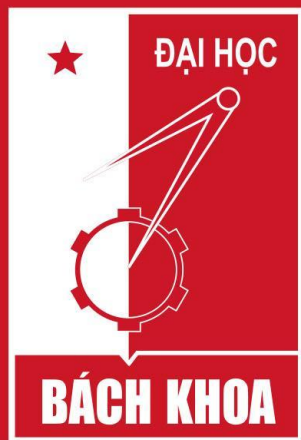




25 YEARS ANNIVERSARY  
SOICT

HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY  
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY



HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY  
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

# PARALLEL ARCHITECTURES

# References

- Michael J. Quinn. **Parallel Computing. Theory and Practice.** McGraw-Hill
- Albert Y. Zomaya. **Parallel and Distributed Computing Handbook.** McGraw-Hill
- Ian Foster. **Designing and Building Parallel Programs.** Addison-Wesley.
- Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar . **Introduction to Parallel Computing, Second Edition.** Addison Wesley.
- Joseph Jaja. **An Introduction to Parallel Algorithm.** Addison Wesley.
- Nguyễn Đức Nghĩa. **Tính toán song song.** Hà Nội 2003.

# Contents

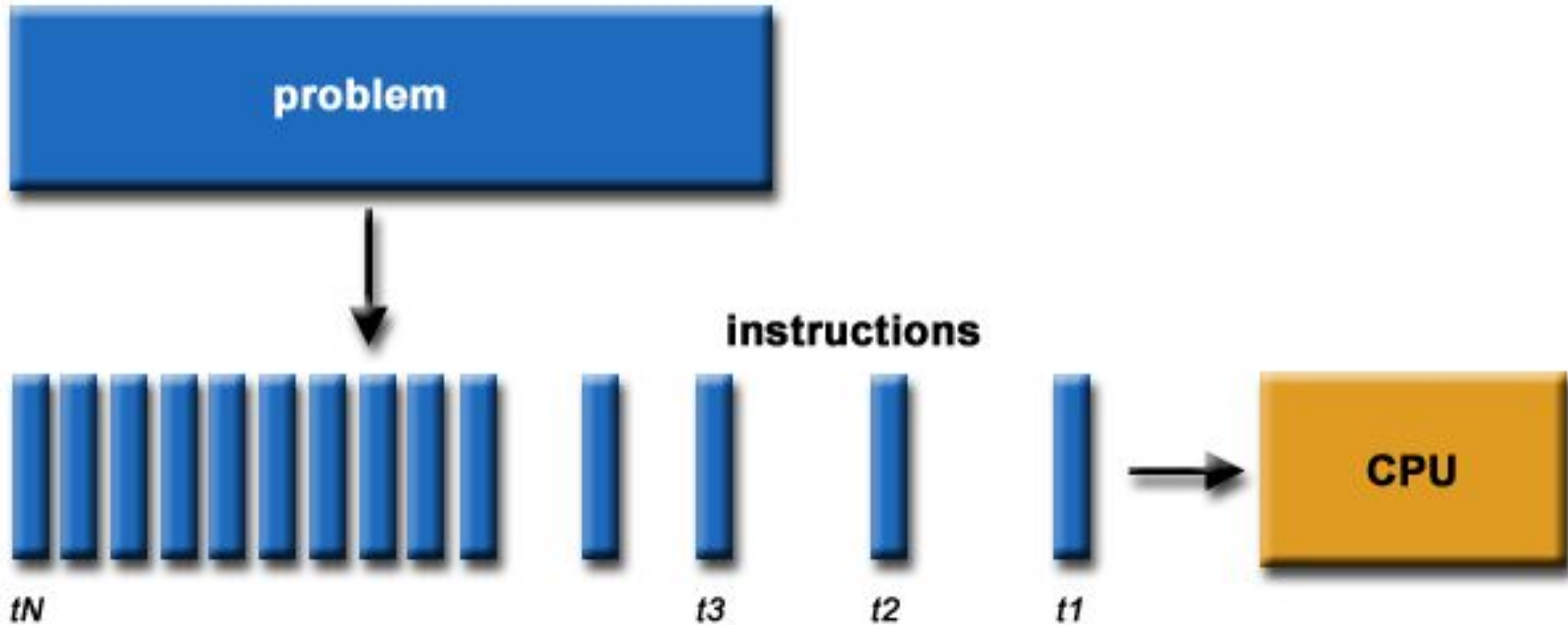
- Parallel Computing Theory
- Parallel Platforms

# 1.1 Parallel Computing Theory

# What is Parallel Computing? (1)

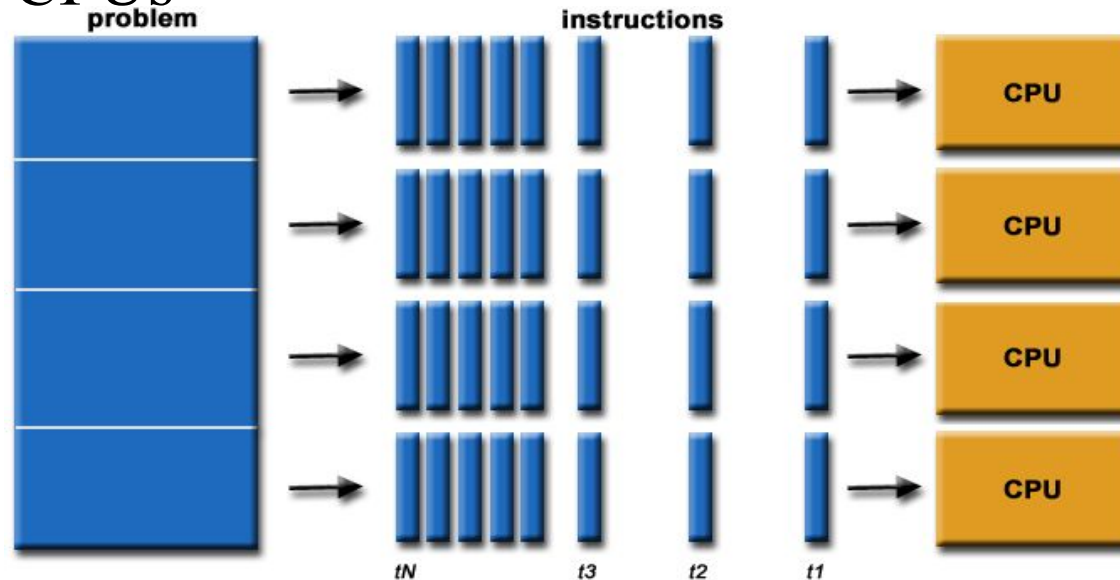
- Traditionally, software has been written for *serial* computation:
  - To be run on a single computer having a single Central Processing Unit (CPU);
  - A problem is broken into a discrete series of instructions.
  - Instructions are executed one after another.
  - Only one instruction may execute at any moment in time.

# What is Parallel Computing? (2)



# What is Parallel Computing? (3)

- In the simplest sense, *parallel computing* is the simultaneous use of multiple compute resources to solve a computational problem.
  - To be run using multiple CPUs
  - A problem is broken into discrete parts that can be solved concurrently
  - Each part is further broken down to a series of instructions
- Instructions from each part execute simultaneously on different CPUs





# Parallel Computing: Resources

- The compute resources can include:
  - A single computer with multiple processors
  - A single computer with (multiple) processor(s) and some specialized computer resources (GPU)
  - An arbitrary number of computers connected by a network
  - A combination of both

# Parallel Computing:

## The computational problem

- The computational problem usually demonstrates characteristics such as the ability to be:
  - Broken apart into discrete pieces of work that can be solved simultaneously;
  - Execute multiple program instructions at any moment in time;
  - Solved in less time with multiple compute resources than with a single compute resource.

# Parallel Computing: what for? (1)

- Parallel computing is an evolution of serial computing that attempts to emulate what has always been the state of affairs in the natural world: many complex, interrelated events happening at the same time, yet within a sequence.
- Some examples:
  - Planetary and galactic orbits
  - Weather and ocean patterns
  - Rush hour traffic in Paris
  - Automobile assembly line
  - Daily operations within a business

# Parallel Computing: what for? (2)

- Traditionally, parallel computing has been considered to be "the high end of computing" and has been motivated by numerical simulations of complex systems and "Grand Challenge Problems" such as:
  - weather and climate
  - chemical and nuclear reactions
  - biological, human genome
  - geological, seismic activity
  - mechanical devices - from prosthetics to spacecraft
  - electronic circuits
  - manufacturing processes

# Parallel Computing: what for? (3)

- Today, commercial applications are providing an equal or greater driving force in the development of faster computers. These applications require the processing of large amounts of data in sophisticated ways. Example applications include:
  - parallel databases, data mining
  - oil exploration
  - web search engines, web based business services
  - computer-aided diagnosis in medicine
  - management of national and multi-national corporations
  - advanced graphics and virtual reality, particularly in the entertainment industry
  - networked video and multi-media technologies

# HPC in film industry

- Special effects are highly compute intensive
- Example: Lord of the Rings
  - company: Weta Digitals
  - 3200 processor cluster
  - a single scene contains:
    - per second 24 frames
    - per frame: 4996 x 3112 points with 32- or 64 bit color encoding
    - each object means a separate compute cycle
- Number of computer-added special effects in movies:
  - Jurassic Park : 75
  - Lord of the Rings (I): 540
  - each of the following episodes of Lord of the Rings doubled the number of special effects
  - last episode of Star-Wars: 2000-2500



# Oregon State University

- Simulate ocean currents: identify causes of global warming.
  - Divide the ocean into:
    - 4096 regions from east to west.
    - 1024 regions from north to south.
    - 12 sea floors.
- ⇒ ~ 50 million blocks in 3D space.
- ⇒ The simulation does ~30 billion calculations in 10 minutes. This work is done continuously throughout the year.

# Weather forecasting

- Divide the atmosphere into 3D space, each block is 1mile x 1mile x 1mile.
- Approximately  $5 \times 10^8$  blocks (cells).
- On each block need to do  $\sim 200$  operations  
 $\Rightarrow$  need to do  $\sim 10^{11}$  operations.
- If you need to forecast for 1 week, 1 minute period we need to do  $10^4$  times,  $10^{11}$  operations each time.
- A normal computer can do:  $10^9$  math operations per second  $\Rightarrow$  it takes  $10^6$  seconds  $\sim 10$  days to do.

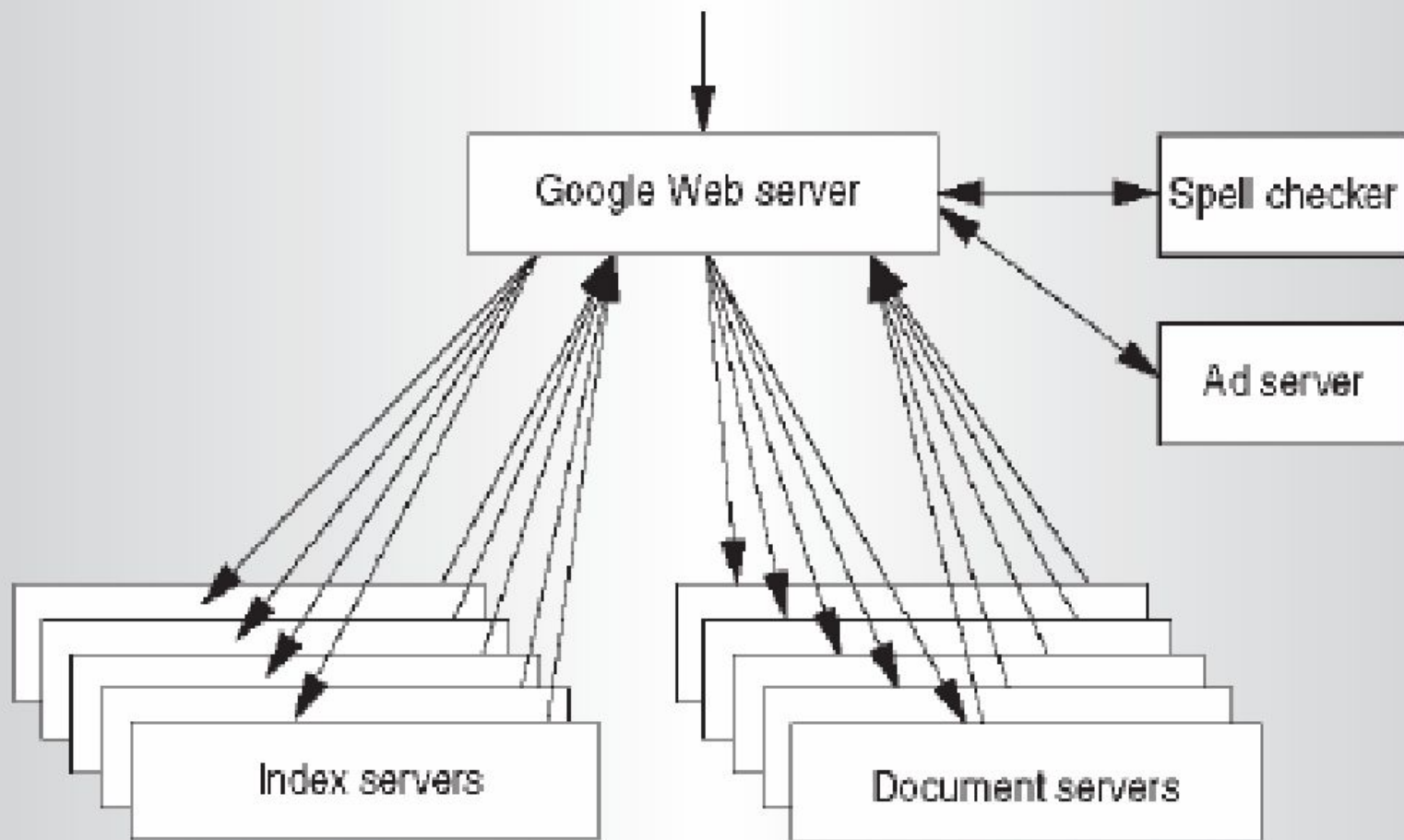


# Simulation of interactions of proteins with water molecules

- Performed on Cray X/MP (~800 million operations/1 second): to simulate 10-12 seconds of protein reaction takes 1 hour.
- Simulating a real reaction on the same Cray X/MP takes 31,688 years.



- each access (=web search) means that several hundreds MB of data have to be touched
- Google's database consists of hundreds of terabytes of data
- A web search is “trivially parallel”
- Google's cluster-philosophy:
  - many cheap (unreliable) nodes
  - Reliability achieved through replication in software and hardware
- End of 2003: several PC cluster with each 15.000 nodes



# Why Parallel Computing? (1)

- This is a legitime question! Parallel computing is complex on any aspect!
- The primary reasons for using parallel computing:
  - Save time - wall clock time
  - Solve larger problems
  - Provide concurrency (do multiple things at the same time)

# Why Parallel Computing? (2)

- Other reasons might include:
  - Taking advantage of non-local resources - using available compute resources on a wide area network, or even the Internet when local compute resources are scarce.
  - Cost savings - using multiple "cheap" computing resources instead of paying for time on a supercomputer.
  - Overcoming memory constraints - single computers have very finite memory resources. For large problems, using the memories of multiple computers may overcome this obstacle.

# Limitations of Serial Computing

- **Limits to serial computing** - both physical and practical reasons pose significant constraints to simply building ever faster serial computers.
- **Transmission speeds** - the speed of a serial computer is directly dependent upon how fast data can move through hardware. Absolute limits are the speed of light (30 cm/nanosecond) and the transmission limit of copper wire (9 cm/nanosecond). Increasing speeds necessitate increasing proximity of processing elements.
- **Limits to miniaturization** - processor technology is allowing an increasing number of transistors to be placed on a chip. However, even with molecular or atomic-level components, a limit will be reached on how small components can be.
- **Economic limitations** - it is increasingly expensive to make a single processor faster. Using a larger number of moderately fast commodity processors to achieve the same (or better) performance is less expensive.

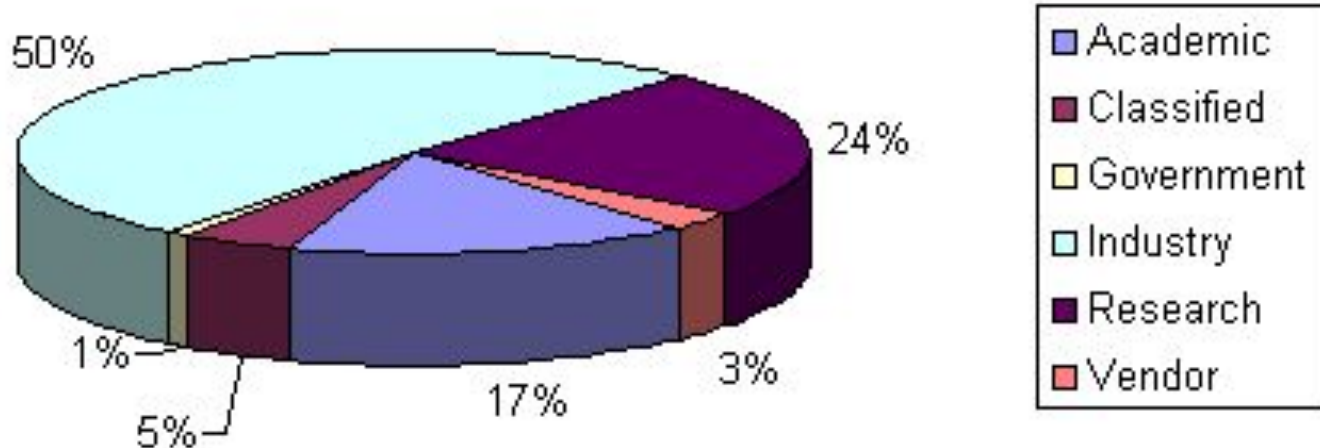
# The future

- During the past 10 years, the trends indicated by ever faster networks, distributed systems, and multi-processor computer architectures (even at the desktop level) clearly show that *parallelism is the future of computing*.
- It will be multi-forms, mixing general purpose solutions (your PC...) and very speciliazed solutions as IBM Cells, ClearSpeed, GPGPU from NVidia ...

# Who and What? (1)

- [Top500.org](http://Top500.org) provides statistics on parallel computing users - the charts below are just a sample.

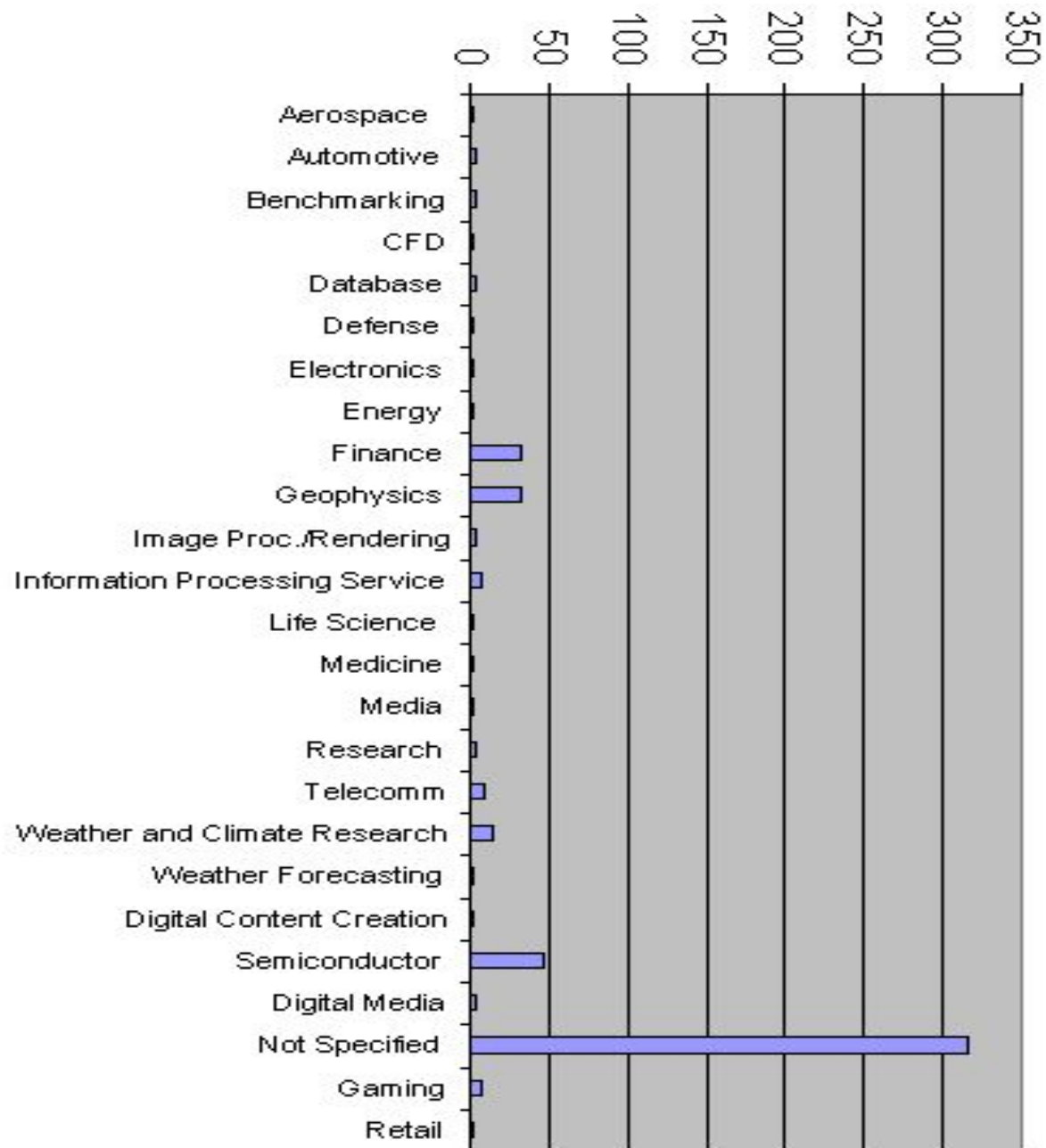
**Who's Doing Parallel Computing?**





# Who and What? (2)

Data obtained from top500.org, June 2006



What Are They Using it For?

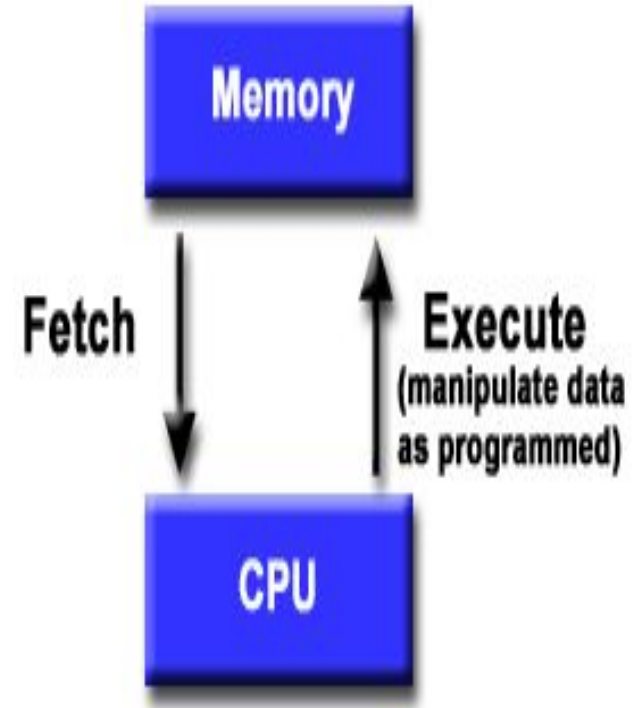
# 1.2 Parallel Platforms

# Von Neumann Architecture

- For over 40 years, virtually all computers have followed a common machine model known as the von Neumann computer.
- A von Neumann computer uses the stored-program concept. The CPU executes a stored program that specifies a sequence of read and write operations on the memory.

# Basic Design

- Basic design
  - Memory is used to store both program and data instructions
  - Program instructions are coded data which tell the computer to do something
  - Data is simply information to be used by the program
- A central processing unit (CPU) gets instructions and/or data from memory, decodes the instructions and then *sequentially* performs them.



# Flynn's Classical Taxonomy

- There are different ways to classify parallel computers. One of the more widely used classifications, in use since 1966, is called Flynn's Taxonomy.
- Flynn's taxonomy distinguishes multi-processor computer architectures according to how they can be classified along the two independent dimensions of *Instruction* and *Data*. Each of these dimensions can have only one of two possible states: *Single* or *Multiple*.

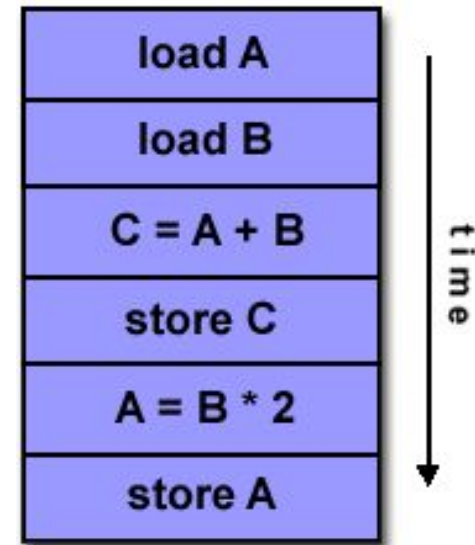
# Flynn Matrix

- The matrix below defines the 4 possible classifications according to Flynn

|                                                     |                                                       |
|-----------------------------------------------------|-------------------------------------------------------|
| <b>S I S D</b><br>Single Instruction, Single Data   | <b>S I M D</b><br>Single Instruction, Multiple Data   |
| <b>M I S D</b><br>Multiple Instruction, Single Data | <b>M I M D</b><br>Multiple Instruction, Multiple Data |

# Single Instruction, Single Data (SISD)

- A serial (non-parallel) computer
- Single instruction: only one instruction stream is being acted on by the CPU during any one clock cycle
- Single data: only one data stream is being used as input during any one clock cycle
- Deterministic execution
- This is the oldest and until recently, the most prevalent form of computer
- Examples: most PCs, single CPU workstations and mainframes

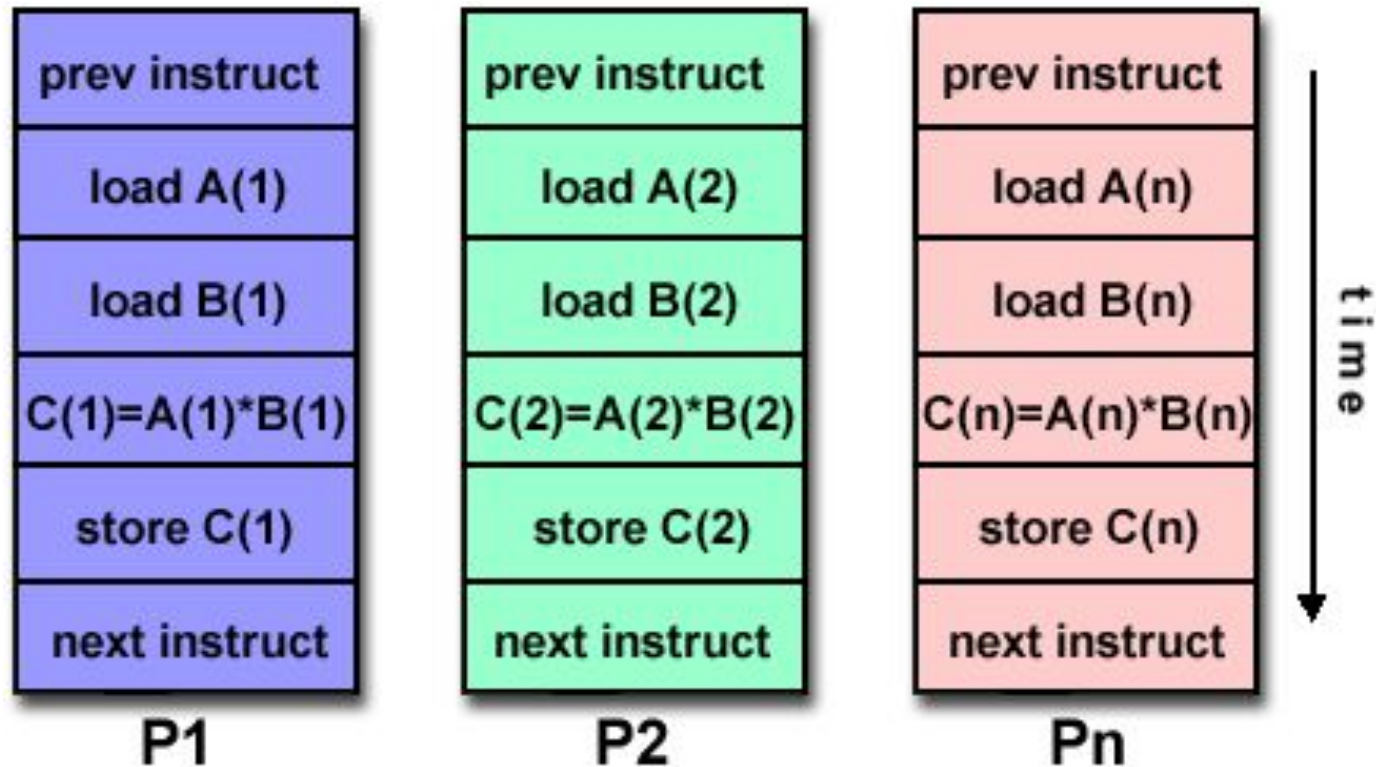


# Single Instruction, Multiple Data (SIMD)

- A type of parallel computer
- Single instruction: All processing units execute the same instruction at any given clock cycle
- Multiple data: Each processing unit can operate on a different data elements
- This type of machine typically has an instruction dispatcher, a very high-bandwidth internal network, and a very large array of very small-capacity instruction units.
- Best suited for specialized problems characterized by a high degree of regularity, such as image processing.
- Synchronous (lockstep) and deterministic execution
- Two varieties: Processor Arrays and Vector Pipelines



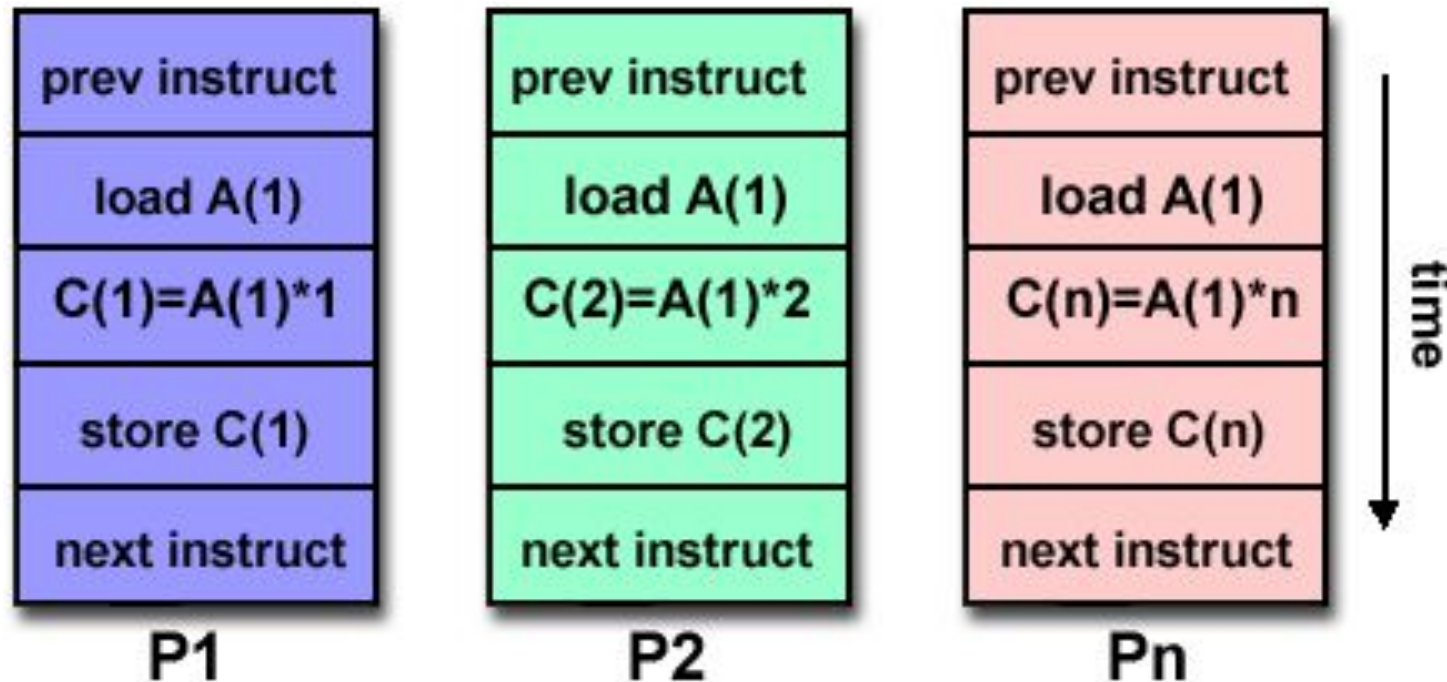
# Single Instruction, Multiple Data (SIMD)



# Multiple Instruction, Single Data (MISD)

- A single data stream is fed into multiple processing units.
- Each processing unit operates on the data independently via independent instruction streams.
- Few actual examples of this class of parallel computer have ever existed. One is the experimental Carnegie-Mellon computer (1971).
- Some conceivable uses might be:
  - multiple frequency filters operating on a single signal stream
  - multiple cryptography algorithms attempting to crack a single coded message.

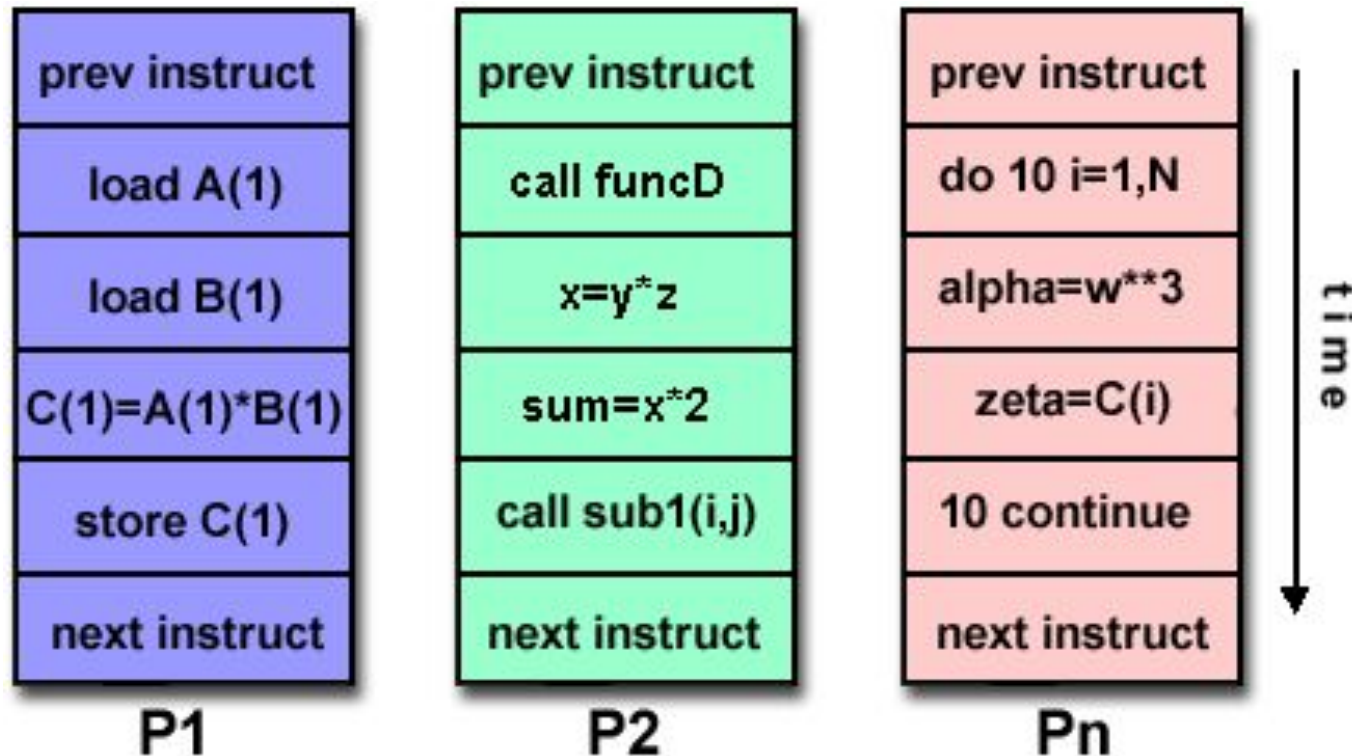
# Multiple Instruction, Single Data (MISD)



# Multiple Instruction, Multiple Data (MIMD)

- Currently, the most common type of parallel computer. Most modern computers fall into this category.
- Multiple Instruction: every processor may be executing a different instruction stream
- Multiple Data: every processor may be working with a different data stream
- Execution can be synchronous or asynchronous, deterministic or non-deterministic
- Examples: most current supercomputers, networked parallel computer "grids" and multi-processor SMP computers - including some types of PCs.

# Multiple Instruction, Multiple Data (MIMD)



# Some General Parallel Terminology(1)

- **Task**

- A logically discrete section of computational work. A task is typically a program or program-like set of instructions that is executed by a processor.

- **Parallel Task**

- A task that can be executed by multiple processors safely (yields correct results)

- **Serial Execution**

- Execution of a program sequentially, one statement at a time. In the simplest sense, this is what happens on a one processor machine. However, virtually all parallel tasks will have sections of a parallel program that must be executed serially.

# Some General Parallel Terminology(2)

- **Parallel Execution**

- Execution of a program by more than one task, with each task being able to execute the same or different statement at the same moment in time.

- **Shared Memory**

- From a strictly hardware point of view, describes a computer architecture where all processors have direct (usually bus based) access to common physical memory. In a programming sense, it describes a model where parallel tasks all have the same "picture" of memory and can directly address and access the same logical memory locations regardless of where the physical memory actually exists.

- **Distributed Memory**

- In hardware, refers to network based memory access for physical memory that is not common. As a programming model, tasks can only logically "see" local machine memory and must use communications to access memory on other machines where other tasks are executing.



# Some General Parallel Terminology(3)

- **Communications**

- Parallel tasks typically need to exchange data. There are several ways this can be accomplished, such as through a shared memory bus or over a network, however the actual event of data exchange is commonly referred to as communications regardless of the method employed.

- **Synchronization**

- The coordination of parallel tasks in real time, very often associated with communications. Often implemented by establishing a synchronization point within an application where a task may not proceed further until another task(s) reaches the same or logically equivalent point.
- Synchronization usually involves waiting by at least one task, and can therefore cause a parallel application's wall clock execution time to increase.



# Some General Parallel Terminology(4)

- **Granularity**

- In parallel computing, granularity is a qualitative measure of the ratio of computation to communication.
- **Coarse:** relatively large amounts of computational work are done between communication events
- **Fine:** relatively small amounts of computational work are done between communication events

- **Observed Speedup**

- Observed speedup of a code which has been parallelized, defined as:  
$$\text{time of serial execution} / \text{time of parallel execution}$$
- One of the simplest and most widely used indicators for a parallel program's performance.

# Some General Parallel Terminology(5)

## • **Parallel Overhead**

- The amount of time required to coordinate parallel tasks, as opposed to doing useful work. Parallel overhead can include factors such as:
  - Task start-up time
  - Synchronizations
  - Data communications
  - Software overhead imposed by parallel compilers, libraries, tools, operating system, etc.
  - Task termination time

## • **Massively Parallel**

- Refers to the hardware that comprises a given parallel system - having many processors. The meaning of many keeps increasing, but currently 6 digits.

# Some General Parallel Terminology(6)

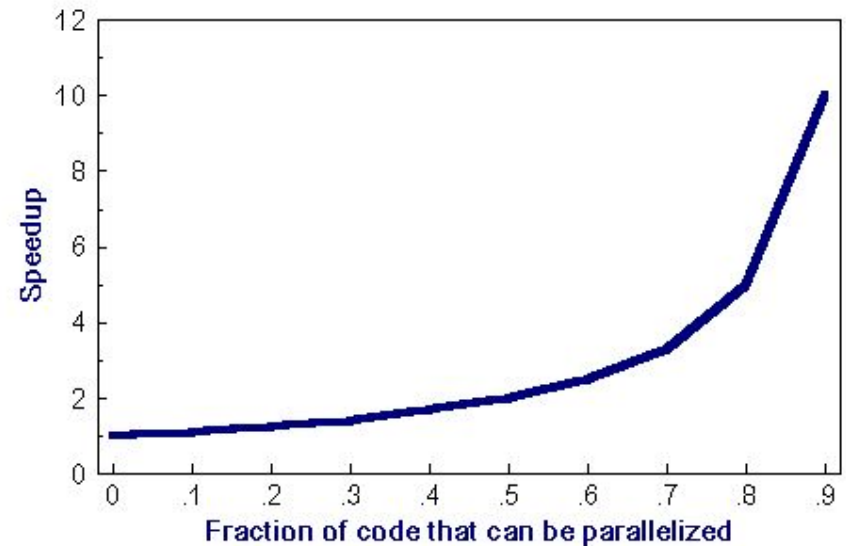
## •Scalability

- Refers to a parallel system's (hardware and/or software) ability to demonstrate a proportionate increase in parallel speedup with the addition of more processors. Factors that contribute to scalability include:
  - Hardware - particularly memory-cpu bandwidths and network communications
  - Application algorithm
  - Parallel overhead related
  - Characteristics of your specific application and coding

# Amdahl's Law

< Amdahl's Law states that potential program speedup is defined by the fraction of code (P) that can be parallelized:

$$\text{speedup} = \frac{1}{1 - P}$$



- If none of the code can be parallelized,  $P = 0$  and the speedup = 1 (no speedup). If all of the code is parallelized,  $P = 1$  and the speedup is infinite (in theory).
- If 50% of the code can be parallelized, maximum speedup = 2, meaning the code will run twice as fast.

# Amdahl's Law

- Introducing the number of processors performing the parallel fraction of work, the relationship can be modeled by

$$\text{speedup} = \frac{1}{\frac{P}{N} + S}$$

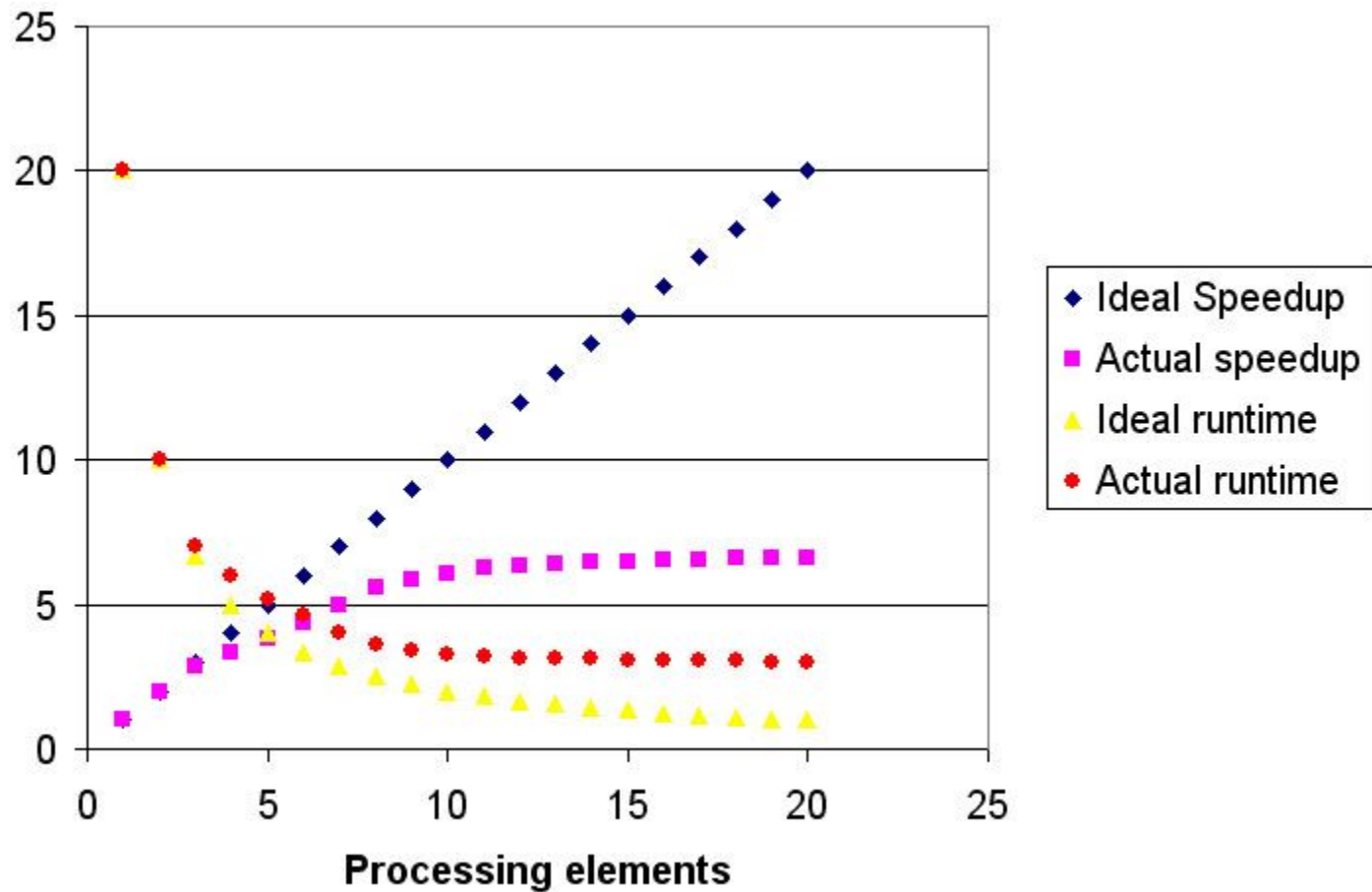
- where P = parallel fraction, N = number of processors and S = serial fraction

# Amdahl's Law

- It soon becomes obvious that there are limits to the scalability of parallelism. For example, at  $P = .50$ ,  $.90$  and  $.99$  (50%, 90% and 99% of the code is parallelizable)

| N     | speedup |         |         |
|-------|---------|---------|---------|
|       | P = .50 | P = .90 | P = .99 |
| 10    | 1.82    | 5.26    | 9.17    |
| 100   | 1.98    | 9.17    | 50.25   |
| 1000  | 1.99    | 9.91    | 90.99   |
| 10000 | 1.99    | 9.91    | 99.02   |

# Amdahl's Law



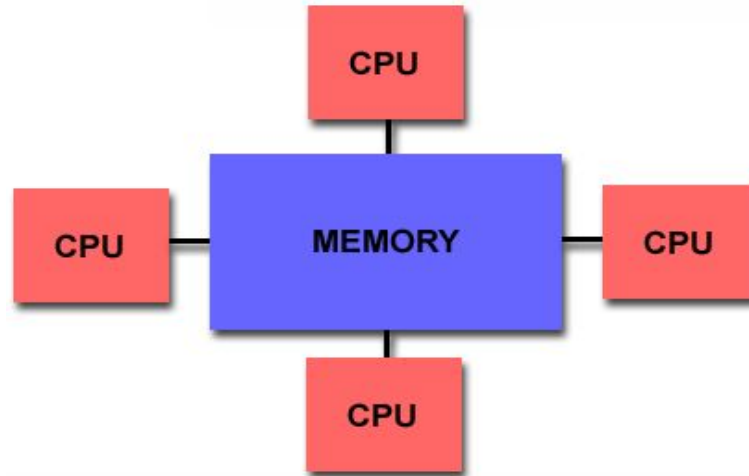
# Memory architectures

- Shared Memory
- Distributed Memory
- Hybrid Distributed-Shared Memory



# Shared Memory

- Shared memory parallel computers vary widely, but generally have in common the ability for all processors to access all memory as global address space.



- Multiple processors can operate independently but share the same memory resources.
- Changes in a memory location effected by one processor are visible to all other processors.
- Shared memory machines can be divided into two main classes based upon memory access times: *UMA* and *NUMA*.

# Shared Memory : UMA vs. NUMA

- Uniform Memory Access (UMA):
  - Most commonly represented today by Symmetric Multiprocessor (SMP) machines
  - Identical processors
  - Equal access and access times to memory
  - Sometimes called CC-UMA - Cache Coherent UMA. Cache coherent means if one processor updates a location in shared memory, all the other processors know about the update. Cache coherency is accomplished at the hardware level.
- Non-Uniform Memory Access (NUMA):
  - Often made by physically linking two or more SMPs
  - One SMP can directly access memory of another SMP
  - Not all processors have equal access time to all memories
  - Memory access across link is slower
  - If cache coherency is maintained, then may also be called CC-NUMA - Cache Coherent NUMA

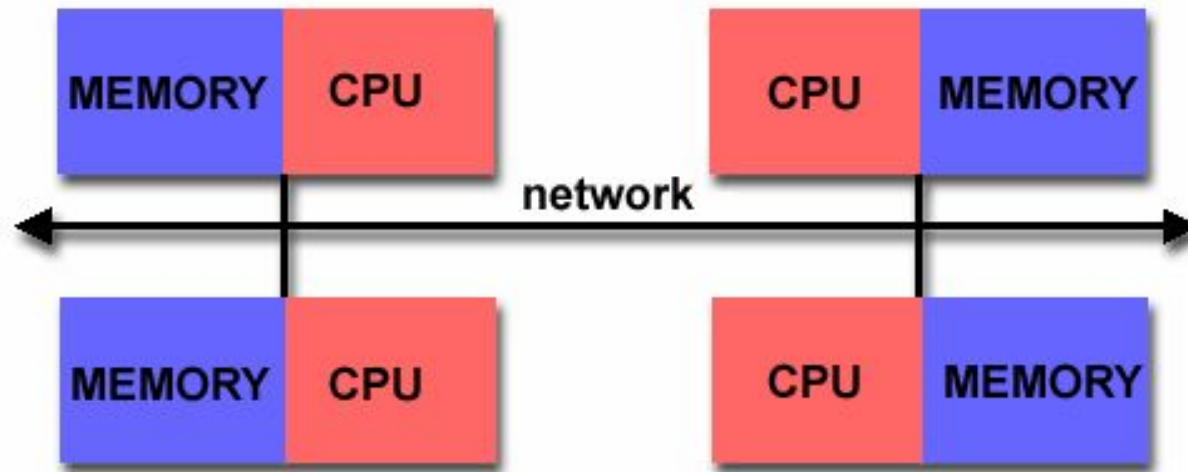
# Shared Memory: Pro and Con

- Advantages
  - Global address space provides a user-friendly programming perspective to memory
  - Data sharing between tasks is both fast and uniform due to the proximity of memory to CPUs
- Disadvantages:
  - Primary disadvantage is the lack of scalability between memory and CPUs. Adding more CPUs can geometrically increase traffic on the shared memory-CPU path, and for cache coherent systems, geometrically increase traffic associated with cache/memory management.
  - Programmer responsibility for synchronization constructs that insure "correct" access of global memory.
  - Expense: it becomes increasingly difficult and expensive to design and produce shared memory machines with ever increasing numbers of processors.

# Distributed Memory

- Like shared memory systems, distributed memory systems vary widely but share a common characteristic. Distributed memory systems require a communication network to connect inter-processor memory.
- Processors have their own local memory. Memory addresses in one processor do not map to another processor, so there is no concept of global address space across all processors.
- Because each processor has its own local memory, it operates independently. Changes it makes to its local memory have no effect on the memory of other processors. Hence, the concept of cache coherency does not apply.
- When a processor needs access to data in another processor, it is usually the task of the programmer to explicitly define how and when data is communicated. Synchronization between tasks is likewise the programmer's responsibility.
- The network "fabric" used for data transfer varies widely, though it can be as simple as Ethernet.

# Distributed Memory



# Distributed Memory: Pro and Con

- Advantages

- Memory is scalable with number of processors. Increase the number of processors and the size of memory increases proportionately.
- Each processor can rapidly access its own memory without interference and without the overhead incurred with trying to maintain cache coherency.
- Cost effectiveness: can use commodity, off-the-shelf processors and networking.

- Disadvantages

- The programmer is responsible for many of the details associated with data communication between processors.
- It may be difficult to map existing data structures, based on global memory, to this memory organization.
- Non-uniform memory access (NUMA) times

# Hybrid Distributed-Shared Memory

## Comparison of Shared and Distributed Memory Architectures

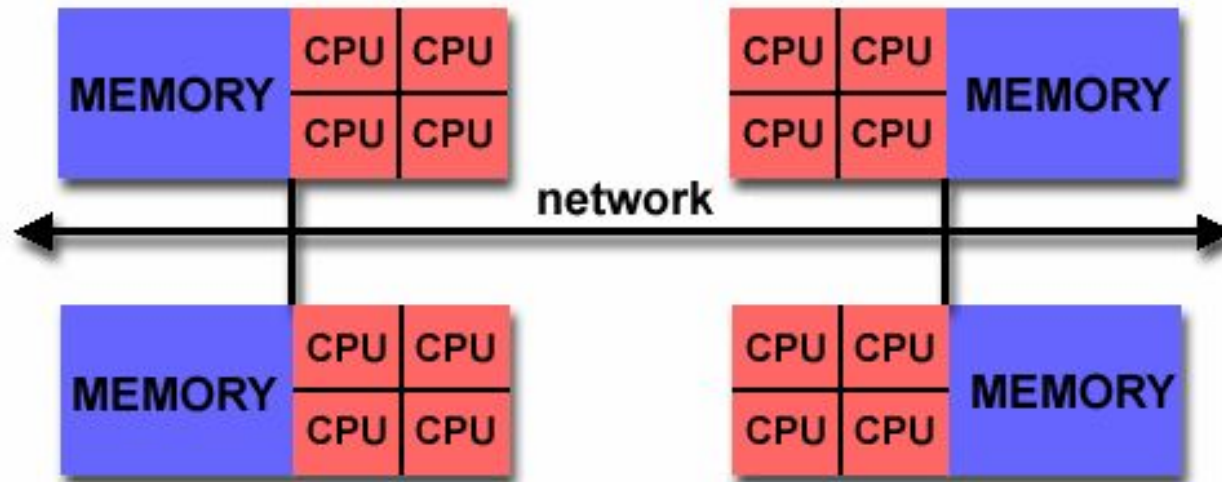
| Architecture                 | CC-UMA                                                        | CC-NUMA                                                                       | Distributed                                                       |
|------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Examples</b>              | SMPs<br>Sun Vexx<br>DEC/Compaq<br>SGI Challenge<br>IBM POWER3 | Bull NovaScale<br>SGI Origin<br>HP Exemplar<br>DEC/Compaq<br>IBM POWER4 (MCM) | Cray T3E<br>Maspar<br>IBM SP2<br>IBM BlueGene                     |
| <b>Communication</b>         | MPI<br>Threads<br>OpenMP<br>shmem                             | MPI<br>Threads<br>OpenMP<br>shmem                                             | MPI                                                               |
| <b>Scalability</b>           | to 10s of procs                                               | to 100s of procs                                                              | to 1000s of procs                                                 |
| <b>Draw Backs</b>            | Memory-CPU bandwidth                                          | Memory-CPU bandwidth<br>Non-uniform access times                              | System administration<br>Programming is hard to develop, maintain |
| <b>Software Availability</b> | many 1000s ISVs                                               | many 1000s ISVs                                                               | 100s ISVs                                                         |

# Hybrid Distributed-Shared Memory

- The largest and fastest computers in the world today employ both shared and distributed memory architectures.
- The shared memory component is usually a cache coherent SMP machine. Processors on a given SMP can address that machine's memory as global.
- The distributed memory component is the networking of multiple SMPs. SMPs know only about their own memory - not the memory on another SMP. Therefore, network communications are required to move data from one SMP to another.
- Current trends seem to indicate that this type of memory architecture will continue to prevail and increase at the high end of computing for the foreseeable future.
- Advantages and Disadvantages: whatever is common to both shared and distributed memory architectures.



# Hybrid Distributed-Shared Memory

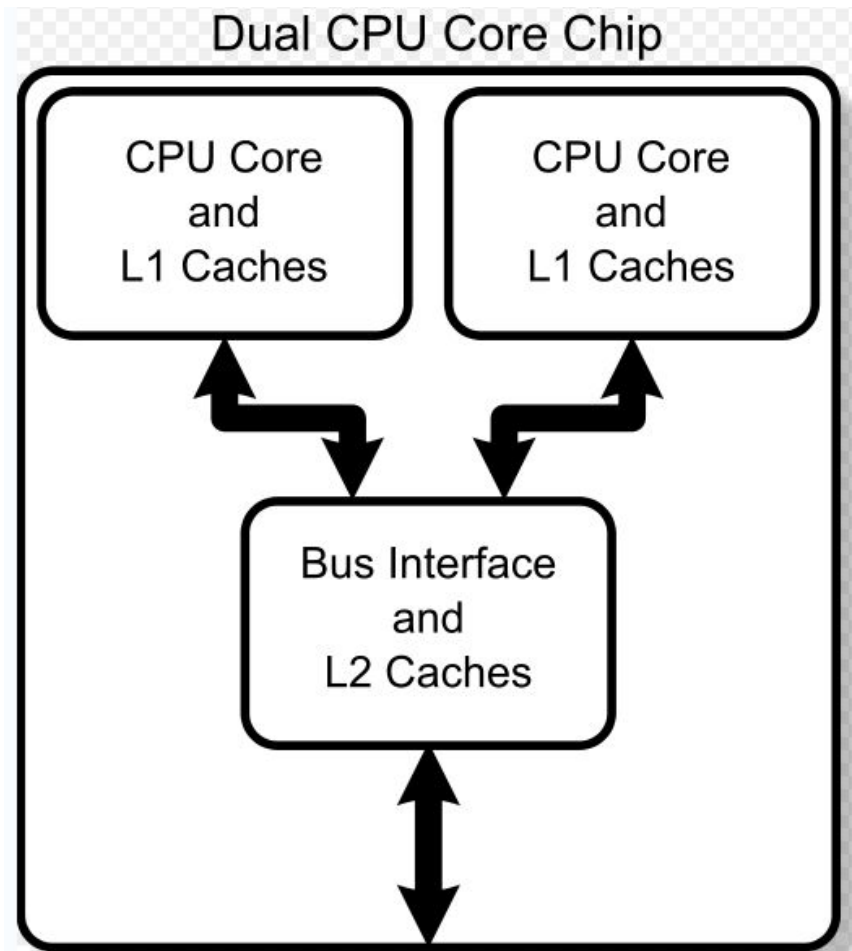


# Some Types of Parallel Computer

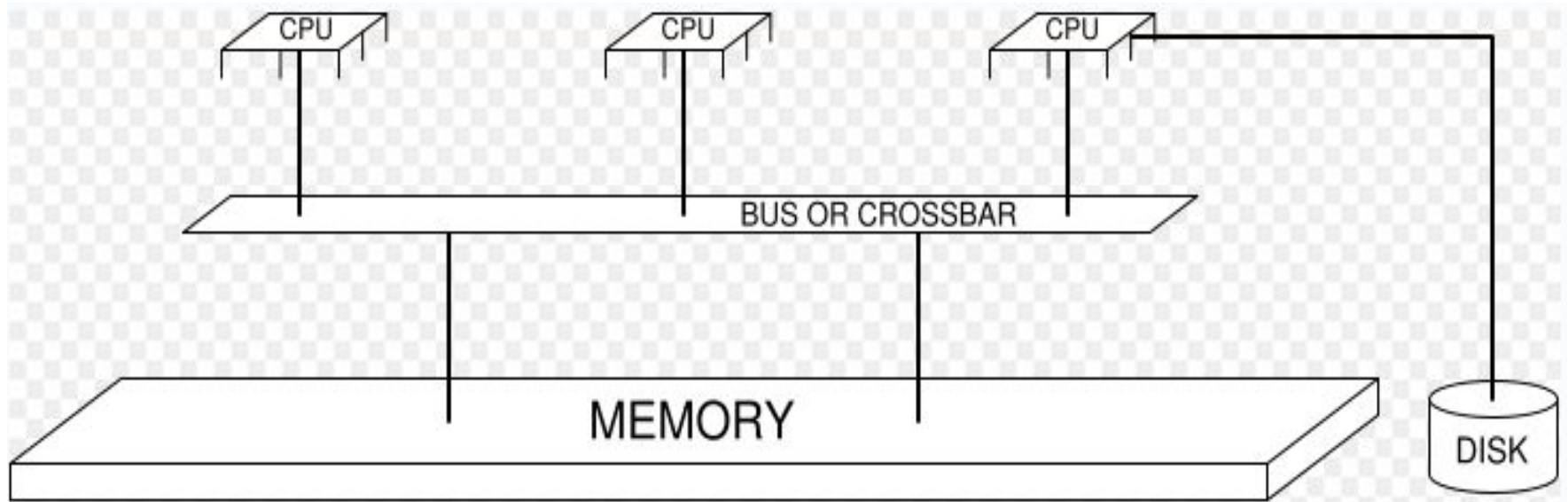
- Multi-core processor
- Symmetric multiprocessing
- Distributed computing

# Multi-core processor

- Multi-cores
- Support Multi-threaded



# Symmetric multiprocessing



- Same CPUs
- Shared Memory
- Each CPU executes a different task.
- Need high-speed bus

# Distributed computing

- Distributed memory system
- CPUs are linked over the network
- Some types:
  - Cluster computing.
  - Massive parallel processing.
  - Grid computing.



25 YEARS ANNIVERSARY  
**SOICT**

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG  
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Thank you  
for your  
attentions  
!



[soict.hust.edu.vn/](http://soict.hust.edu.vn/)



[fb.com/groups/soict](https://fb.com/groups/soict)

